

The Transformer Network for the Dial-a-Ride Problem

Lucas Gruaz, Aziz Laadhar, Aoyu Gong, Benedek Harsanyi
École Polytechnique Fédérale de Lausanne (EPFL)
{lucas.gruaz, aziz.laadhar, aoyu.gong, benedek.harsanyi}@epfl.ch

May 6, 2024

Abstract

The Dial-a-Ride problem (DARP) is a transportation problem, involving pickup and delivery requests, that has traditionally been solved using mixed integer programming. This paper explores the application of deep learning techniques to solve the DARP efficiently. A deep learning framework based on a variant of the Transformer architecture is proposed and trained using supervised and reinforcement learning methods. The model's performance is evaluated through numerical experiments, comparing its solutions to those of a recent Gurobi-based solver in terms of cost and constraint violations. The results demonstrate the potential of neural networks to address NP-hard problems like the DARP, although certain limitations and opportunities for further improvement are acknowledged.

dial-a-ride problem, transformer, transportation, vehicle routing, reinforcement learning

1 Introduction

The Dial-a-Ride problem (DARP) is an optimization problem related to transportation. It revolves around efficiently scheduling pick-ups and drop-offs for a fleet of vehicles operating in a specific geographical area. The objective is to minimize the total travel time for the fleet while also satisfying the users' demand. The DARP finds applications in various real-world scenarios such as ride-sharing services, public transit systems and logistics companies. The complexity of the constraints involved, including factors such as time windows, capacity limits, and distance constraints, poses significant challenges when solving the DARP. Therefore, the development of effective algorithms to tackle this problem is a subject of active research in the domain of transportation and logistics. The DARP can be considered an extension of other well-known optimization problems like the Traveling Salesman Problem (TSP). The TSP seeks to find the shortest route that visits each city only once and returns to the starting point. Similarly,

the DARP can also be viewed as a variant of the Vehicle Routing Problem (VRP). The VRP aims to determine optimal routes for a fleet of vehicles serving multiple locations.

The rest of the paper is organized as follows: Subsection 1.1 provides a literature review, examining previous research on the DARP. Section 2 outlines the problem statement, defining the specific objectives and constraints as a mixed integer programming task. In Section 3, we represent the DARP as a Markov decision process (MDP). Section 4 introduces a deep learning framework, based on a variant of the Transformer architecture, for solving the problem. We evaluate the proposed approach through numerical experiments in Section 5, employing both supervised and reinforcement learning techniques. Finally, we discuss and summarize the main conclusions of the paper in the last sections.

1.1 Literature Review

Most of the literature in this domain primarily concentrates on static and deterministic formulations of the DARP, wherein the problem remains unchanged over time and the data used for planning is assumed to be certain. The branch-and-cut (B&C) algorithm was successfully applied by [1]. This algorithm adds cutting planes to the problem in the branch-and-bound tree, which helps tighten the LP-relaxations and increase the chances of finding integer solutions. [2] introduced a B&C-based solution that handles both ordinary and dynamic time windows in a column-generation subproblem. In 2021, [3] proposed a new formulation for the DARP, introducing the notion of "restricted fragments", which are specific segments of routes that can effectively express any DARP route. By enumerating these restricted fragments and establishing domination relationships between them, they presented a formulation that can be solved with a B&C algorithm.

Since the DARP is a NP-hard problem, exact solvers tend to focus on small-sized instances. Many heuristic-based solutions have been developed to overcome this issue. The large neighborhood search (LNS) approach [4] involves iteratively modifying the current solution by removing a portion of it and creating a new complete solution using the remaining elements. Simulated annealing (SA), a metaheuristic approach, generates new solutions by selecting neighbors of the current solution in a stochastic manner, as described by [5].

Over the past few years, deep learning techniques found successful applications to NP-hard combinatorial problems. For instance, in [6], the Transformer architecture originally designed for natural language processing (NLP) was adapted as a policy network for constructing routes in an autoregressive manner for the Traveling Salesman Problem (TSP).

2 Problem Statement

The DARP involves the optimization of vehicle routes and schedules for a set of users with specific transportation requests. Each user, indexed as $n \in \mathcal{N} =$

$\{1, 2, \dots, N\}$, require transportation from a pick-up location p_n to a drop-off location d_n . The transportation service is provided by a group of identical vehicles, indexed as $k \in \mathcal{K} = \{1, 2, \dots, K\}$. These vehicles depart from a common source station s^+ and must finish their route at a destination station s^- . The objective of DARP is to design a set of minimum-cost vehicle routes that efficiently serve all the users, while adhering to various constraints.

These constraints include a maximum route duration T , maximum vehicle capacity Q , and a maximum ride time L for each user. The set of all locations can be written as $\mathcal{V} = \mathcal{P} \cup \mathcal{D} \cup \{s^+, s^-\}$, where $\mathcal{P}$ is the set of all pick-up locations, $\mathcal{D}$ is the set of all drop-off locations, s^+ is the source station and s^- is the destination station. Each location i has coordinates (x_i, y_i) and is associated with a service duration δ_i and a load q_i . The service duration is zero for the source and destination stations, and such that $\delta_{p_n} = \delta_{d_n} > 0$ for each user n . The load satisfies $q_{p_n} = -q_{d_n} > 0$ for each user n . Additionally, each location has a time window $[e_i, l_i]$ representing the earliest and latest times for service to start at that location.

Users can either specify the time window for their pick-up location or their drop-off location, but not both. If a user specifies the time window for the pick-up location, they are referred to as an inbound user. Otherwise, they are an outbound user. It's important to note that although a user only specifies one time window, the other time window can be constrained accordingly [1].

We can define the DARP by considering each location as a vertex in a complete directed graph $G = (\mathcal{V}, \mathcal{E})$. The set of vertices, $\mathcal{V}$, remains the same as described earlier. The set of edges, $\mathcal{E} = \{i, j \mid i, j \in \mathcal{V}\}$, represents all possible connections between the vertices. The routing cost $c_{i,j}$ and the travel time $t_{i,j}$ of each edge $\{i, j\} \in \mathcal{E}$ are determined based on the Euclidean distance and the service duration, i.e.,

$$c_{i,j} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2},$$

$$t_{i,j} = c_{i,j} + \delta_i.$$

The travel time consists of two components: the service duration at location i and the time taken from location i to location j [3]. To mathematically represent the DARP, we introduce decision variables for each vehicle $k \in \mathcal{K}$, each location $i \in \mathcal{V}$, each edge $\{i, j\} \in \mathcal{E}$, and/or each user $n \in \mathcal{N}$. The decision variables are defined as follows:

- Let $\phi_{i,j}^k = 1$ if vehicle k takes edge $\{i, j\}$, and $\phi_{i,j}^k = 0$ otherwise.
- Let B_i^k denote the time at which vehicle k starts a service at location i .
- Let Q_i^k represent the load of vehicle k when leaving location i .
- Let L_n^k denote the ride time of user n on vehicle k .

As mentioned earlier, we start by tightening the time windows for each user. For an inbound user n specifying the pick-up time window $[e_{p_n}, l_{p_n}]$, the time

window for its drop-off location can be determined as

$$\begin{aligned} e_{d_n} &= \max(0, e_{p_n} + \delta_{p_n} + t_{p_n, d_n}), \\ l_{d_n} &= l_{p_n} + \delta_{p_n} + L. \end{aligned}$$

For an outbound user n' specifying the drop-off time window $[e_{d_{n'}}, l_{d_{n'}}]$, the time window for its pick-up location can be determined as

$$\begin{aligned} e_{p_{n'}} &= \max(0, e_{d_{n'}} - L - \delta_{p_{n'}}), \\ l_{p_{n'}} &= l_{d_{n'}} - t_{p_{n'}, d_{n'}} - \delta_{p_{n'}}. \end{aligned}$$

With these definitions, we can follow [1, Sec. 3] and formulate the DARP as a mixed-integer program as follows:

$$\text{minimize} \quad \sum_{k \in \mathcal{K}} \sum_{i \in \mathcal{V}} \sum_{j \in \mathcal{V}} c_{i,j} \phi_{i,j}^k, \quad (1)$$

$$\text{subject to} \quad \sum_{k \in \mathcal{K}} \sum_{j \in \mathcal{V}} \phi_{i,j}^k = 1, \quad \forall i \in \mathcal{P}, \quad (2)$$

$$\sum_{j \in \mathcal{V}} \phi_{p_n, j}^k - \sum_{j \in \mathcal{V}} \phi_{d_n, j}^k = 0, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (3)$$

$$\sum_{j \in \mathcal{V}} \phi_{s^+, j}^k = 1, \quad \forall k \in \mathcal{K}, \quad (4)$$

$$\sum_{j \in \mathcal{V}} \phi_{i, j}^k - \sum_{j \in \mathcal{V}} \phi_{j, i}^k = 0, \quad \forall i \in \mathcal{P} \cup \mathcal{D}, \quad \forall k \in \mathcal{K}, \quad (5)$$

$$\sum_{i \in \mathcal{V}} \phi_{i, s^-}^k = 1, \quad \forall k \in \mathcal{K}, \quad (6)$$

$$B_j^k \geq (B_i^k + t_{i,j}) \phi_{i,j}^k, \quad \forall i \in \mathcal{V}, \quad \forall j \in \mathcal{V}, \quad \forall k \in \mathcal{K}, \quad (7)$$

$$Q_j^k \geq (Q_i^k + q_j) \phi_{i,j}^k, \quad \forall i \in \mathcal{V}, \quad \forall j \in \mathcal{V}, \quad \forall k \in \mathcal{K}, \quad (8)$$

$$L_n^k = B_{d_n}^k - (B_{p_n}^k + \delta_{p_n}), \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (9)$$

$$B_{s^-}^k - B_{s^+}^k \leq T, \quad \forall k \in \mathcal{K}, \quad (10)$$

$$e_i \leq B_i^k \leq l_i, \quad \forall i \in \mathcal{V}, \quad \forall k \in \mathcal{K}, \quad (11)$$

$$t_{p_n, d_n} - \delta_{p_n} \leq L_n^k \leq L, \quad \forall n \in \mathcal{N}, \quad \forall k \in \mathcal{K}, \quad (12)$$

$$\max(0, q_i) \leq Q_i^k \leq \min(Q, Q + q_i), \quad \forall i \in \mathcal{V}, \quad \forall k \in \mathcal{K}, \quad (13)$$

$$\phi_{i,j}^k \in \{0, 1\}, \quad \forall i \in \mathcal{V}, \quad \forall j \in \mathcal{V}, \quad \forall k \in \mathcal{K}. \quad (14)$$

The objective function (1) aims to minimize the sum of the routing costs. Equalities (2) and (3) ensure that each location is visited exactly once and that the pick-up and drop-off locations of each user are visited by the same vehicle. Equalities (4), (5), and (6) ensure that the route of each vehicle starts at the source station and finishes at the destination station. Inequalities (7) and (8) guarantee that time and load variables are consistent. The ride time of each user is defined in equality (9) and is bounded by inequalities (12). Inequality

(10) ensures that the route duration of each route does not exceed the maximum allowed duration. Inequality (11) establishes the time windows constraints. Inequality (13) establishes the vehicle capacity constraints.

3 Markov Decision Process

In this section, we initially present the DARP as a Markov decision process (MDP) by relaxing Constraints (10), (11), and (12). Based on the original formulation and the MDP formulation, we then design the inputs and outputs of the proposed architecture.

3.1 MDP Formulation

Based on Section 2, we define a MDP by introducing the following definitions.

1. States: At step τ , let $s_\tau = (u_\tau, v_\tau)$ represent the state of the system, where $u_\tau = (u_\tau^1, u_\tau^2, \dots, u_\tau^N)$ is the state of the N users and $v_\tau = (v_\tau^1, v_\tau^2, \dots, v_\tau^K)$ is the state of the K vehicles. The state of user $n \in \mathcal{N}$ is defined as

$$u_\tau^n = ((x_{p_n}, y_{p_n}), (x_{d_n}, y_{d_n}), \delta_{p_n}, q_{p_n}, [e_{p_n}, l_{p_n}], [e_{d_n}, l_{d_n}]),$$

and the state of vehicle $k \in \mathcal{K}$ is expressed as $v_\tau^k = (r_\tau^k, b_\tau^k, f_\tau^k)$. Here, r_τ^k denotes the path of vehicle k consisting of a set of locations, i.e.,

$$r_\tau^k = \{s^+, i_1, i_2, \dots, i_{|r_\tau^k|-1}\},$$

where $|r_\tau^k|$ is the number of locations visited by the vehicle k . b_τ^k is the schedule of vehicle k , which consists of a set of start time, i.e.,

$$b_\tau^k = \{B_{s^+}^k, B_{i_1}^k, B_{i_2}^k, \dots, B_{i_{|r_\tau^k|-2}}^k\},$$

where $|b_\tau^k| = |r_\tau^k| - 1$ since the vehicle has not started a service at location $i_{|r_\tau^k|-1}$. f_τ^k is the next available time of vehicle k . Note that, at each step, the ride time of all users and the loads of all vehicles can be derived from the states.

2. Actions: At each step, the vehicle that has the smallest next available time, denoted by κ_τ , performs an action. In case of ties, the vehicle with the smallest index is selected. Let a_τ be the action of the system at step τ . If $s_\tau = s$ and $\kappa_\tau = \kappa$, we have

$$a_\tau \in \begin{cases} \mathcal{P}_{s,\kappa} \cup \mathcal{D}_{s,\kappa} \cup \{s^-\}, & \text{if vehicle } \kappa \text{ starts a service at location } i_{|r_\tau^\kappa|-1} \text{ and moves to location } a_\tau, \\ \mathcal{W}, & \text{if vehicle } \kappa \text{ waits for } a_\tau \text{ time units at location } i_{|r_\tau^\kappa|-1}, \end{cases}$$

where $\mathcal{P}_{s,\kappa}$ represents the set of unvisited pick-up locations where vehicle κ can pick up the corresponding users, and $\mathcal{D}_{s,\kappa}$ denotes the set of unvisited

drop-off locations associated with pick-up locations that have already been visited by vehicle κ . $\mathcal{W}$ is supposed to contain any possible waiting amount, but in order to transfer action prediction into a classification problem, $\mathcal{W}$ has been set to $\{\tilde{w}\}$ where $\tilde{w}$ is a small fixed constant. A vehicle can wait multiple times in a row to approximate any waiting amount. Here, $\mathcal{P}_{s,\kappa}$ and $\mathcal{D}_{s,\kappa}$ can be expressed as

$$\begin{aligned}\mathcal{P}_{s,\kappa} &= \{p_n \mid p_n \in \mathcal{P} \setminus \cup_{k=1}^K(r_\tau^k), \quad Q_{i_{|r,\kappa|-1}}^\kappa + q_{p_n} \leq Q, n \in \mathcal{N}\} \\ \mathcal{D}_{s,\kappa} &= \{d_n \mid p_n \in r_\tau^\kappa, \quad d_n \notin r_\tau^\kappa, n \in \mathcal{N}\}.\end{aligned}$$

3. State Transition Function: At step τ , with $s_\tau = s$ and $\kappa_\tau = \kappa$, the state transition function gives the state of the system at step $\tau + 1$, denoted as $s_{\tau+1}$. For vehicle κ , there are three cases to consider.

- 1) If $a_\tau \in \mathcal{P}_{s,\kappa} \cup \mathcal{D}_{s,\kappa}$, then

$$\begin{aligned}r_{\tau+1}^\kappa &= r^\kappa \cup \{a_\tau\}, \\ b_{\tau+1}^\kappa &= b^\kappa \cup \{f^\kappa\}, \\ f_{\tau+1}^\kappa &= \max\left(f^\kappa + t_{i_{|r,\kappa|-1}, a_\tau}, e_{a_\tau}\right).\end{aligned}$$

- 2) If $a_\tau = s^-$, then

$$\begin{aligned}r_{\tau+1}^\kappa &= r^\kappa \cup \{s^-\}, \\ b_{\tau+1}^\kappa &= b^\kappa \cup \{f^\kappa, f^\kappa + t_{i_{|r,\kappa|-1}, s^-}\}, \\ f_{\tau+1}^\kappa &= 2T.\end{aligned}$$

- 3) If $a_\tau \in \mathcal{W}$, then

$$\begin{aligned}r_{\tau+1}^\kappa &= r^\kappa, \\ b_{\tau+1}^\kappa &= b^\kappa, \\ f_{\tau+1}^\kappa &= f^\kappa + \tilde{w}.\end{aligned}$$

Additionally, the state of the users and other vehicles remains unchanged, i.e. $u_{\tau+1}^n = u^n$ for each user $n \in \mathcal{N}$ and $v_{\tau+1}^k = v^k$ for each vehicle $k \in \mathcal{K} \setminus \{\kappa\}$. The MDP will be terminated when all vehicles arrive at the destination station, i.e. when the smallest next available time equals $2T$.

4. Rewards: When vehicle κ takes action a in state s at step τ , the reward $R_\tau(s, a)$ is defined as

$$R_\tau(s, a) = \begin{cases} c_{i_{|r,\kappa|-1}, a}, & \text{if } a \in \mathcal{P}_{s,\kappa} \cup \mathcal{D}_{s,\kappa} \cup \{s^-\}, \\ 0, & \text{if } a \in \mathcal{W}. \end{cases}$$

With this definition, the goal would be to achieve a minimal reward (cost). Since we consider that constraints may be violated, penalties for constraint violations should be included in the reward calculation. The specific implementation of the cost and penalties for constraint violation are described in more details in section 5.4.2.

3.2 Input representation

The state described in 3.1 is not used directly for action selection. Instead, an input is generated to be fed into a prediction model. The input consists of a graph where nodes correspond to locations and edges capture information about each pair of nodes. Additionally, there is an extra node w representing the waiting action, located at the same coordinates as the vehicle about to perform an action. The intuition behind this input representation is that the vehicle must choose an action by selecting one of its neighbor in the graph (a pick-up/drop-off location, the destination station or the waiting node). Once the action is selected, the effects of the action can be applied by simulating a step of the MDP. This would yield a new state, and the new corresponding input can be computed again, generating a new graph with updated vehicle location and other relevant information.

More concretely, each node $i \in \mathcal{P} \cup \mathcal{D} \cup \{s_-, s_+, w\}$ will contain the following features α_i :

- t_i : Node type (pick-up, drop-off, source, destination, waiting).
- (x_i, y_i) : Coordinates of the location.
- $[e_i, l_i]$: Time window of the location.
- δ_i : Service time of the location.
- L_i : Remaining ride time of the user at the location. If i is a drop-off location and the corresponding user is in a vehicle, L_i is equal to the difference between the maximum ride time L and the current ride time of the user.
- k_i : Presence of a vehicle, equal to 1 if a vehicle is present on the location, 0 otherwise.
- Q_i : Vehicle free capacity, 0 if $k_i = 0$.
- f_i : Vehicle next available time, 0 if $k_i = 0$.
- isNext_i : equal to 1 if the present vehicle is the one performing the next action, 0 otherwise.

We assume that each node can only contain one vehicle and one user. If multiple vehicles are at the same location, e.g on source station, we duplicate the corresponding node. It is important to note that a user can only be served by a single vehicle, which avoids any potential issues resulting from the duplication process. Each edge (i, j) will contain the following features β_{ij} :

- d_{ij} : Distance between node i and node j .

- p_{ij} : Pairing information, equal to 1 if i and j are pick-up and drop-off locations of the same user, 0 otherwise. Two such nodes possess a significant connection to one another. It appears beneficial to capture this special relationship.
- w_{ij} : Waiting information, equal to 1 if i or j is the waiting node, 0 otherwise. The waiting node carry a different meaning than the other nodes. Including this information in the edges seems important.

While there exists a clear distinction between the *state*, denoting the internal status of the DARP as described in the MDP formulation 3.1, and the *input*, representing the graph derived from the state that is utilized for prediction, the term "state" may be employed to refer to the input when the context unambiguously implies its intended meaning. This usage aligns with the conventions found in relevant literature on topics such as reinforcement learning.

3.3 Edge removal

By default, every pair of node in the input graph is connected by an edge. In this section, a pruning procedure is described to reduce the graph complexity by removing unnecessary edges. This ensures that some constraints will be respected. The following principles are followed for eliminating edges:

- Nodes that have already been visited should not be connected to any other node. Once a node has been visited, there is no interest in revisiting it at a later stage, and the node should not be taken into consideration for further planning.
- Vehicles should only be connected to acceptable actions. For instance, a vehicle **should not** be connected to:
 - A location where an other vehicle is present.
 - Pick-up locations if the capacity constraint is violated.
 - Drop-off locations for users who are not onboard.
 - The waiting node if the vehicle is on a drop-off location and the time window at the current location is open. Waiting on a drop-off node when the time window is open serves no purpose. Instead, the vehicle should serve the user at that location and go (possibly wait) somewhere else. Removing these edges reduces unnecessary complexity and simplifies the vehicle's decision.
 - The waiting node if the vehicle is on a pickup location and waiting would break the user's time window.

An example of a simple situation with a few step of MDP is shown in Figure 1.

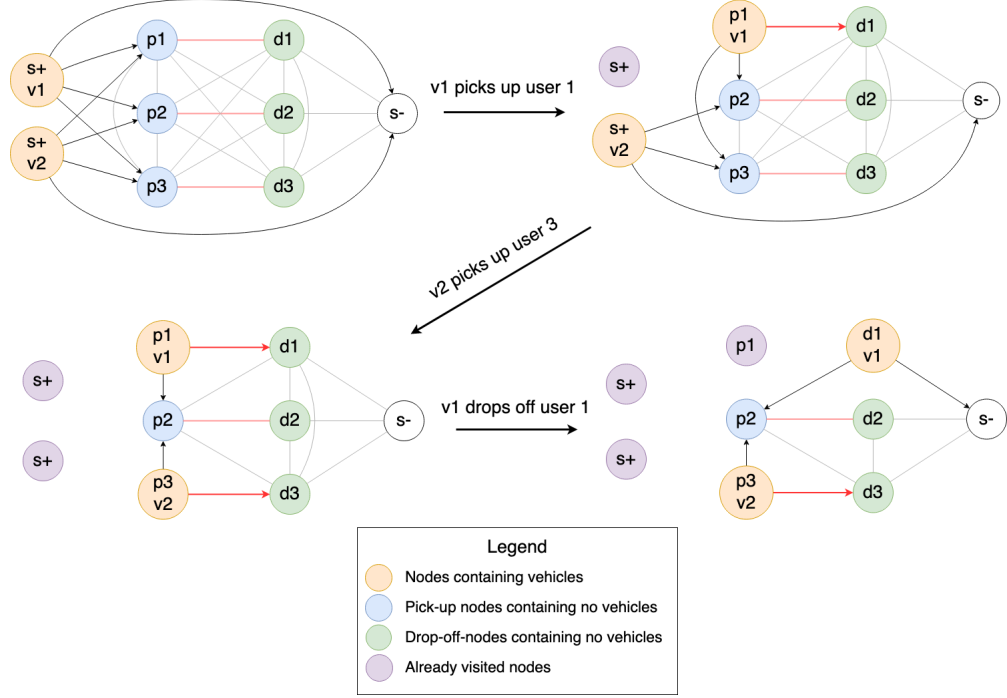


Figure 1: Input representation of a simple situation with three users, showing the first three steps of MDP. The waiting node, representing the waiting action, is not shown here for clarity. It would be connected to every other node that is not already visited. Arrows represent available actions for each vehicle. At each step, the vehicle with the smallest next available time performs the action. Note that there is a separate source node for each vehicle, since node features can only capture information about one vehicle. This is the only situation where multiple vehicles can occupy the same location simultaneously.

4 Proposed architecture

As designed in section 3.2, the input consists of a graph where each node represents a location and contains information about the user related to the location (if any) and the vehicle present on the location (if any). The architecture of the model used to process the input and output a probability distribution over the actions is shown in Figure 2 and will be described in more details in the rest of this section.

4.1 Node embeddings

The first step is to convert nodes and edges representations to vectors of dimension d_{model} . This is done by a simple linear layer. For each node i with features $\alpha_i \in \mathbb{R}^{d_\alpha}$ we have:

$$h_i^0 = A\alpha_i + a \in \mathbb{R}^{d_{\text{model}}}, \quad A \in \mathbb{R}^{d_{\text{model}} \times d_\alpha}, \quad a \in \mathbb{R}^{d_{\text{model}}}$$

The weights and biases are shared among all nodes. For each edge (i, j) with edge features $\beta_{ij} \in \mathbb{R}^{d_e}$ we have:

$$e_{ij}^0 = B\beta_{ij} + b \in \mathbb{R}^{d_{\text{model}}}, \quad B \in \mathbb{R}^{d_{\text{model}} \times d_e}, \quad b \in \mathbb{R}^{d_{\text{model}}}$$

The weights and biases are shared among all edges. We use Laplacian eigenvectors as positional encoding. Since the sign of the eigenvectors is arbitrary, we randomly flip it, following [7], [8]. The Laplacian positional encoding is added to the node embedding.

4.2 Graph transformer

Graph transformers were first introduced in [8]. The main idea behind graph transformers is to extend the success of transformer models [9], originally designed for sequential data like text, to graph-structured data. Transformers excel at capturing long-range dependencies in sequential data through self-attention mechanisms. Graph transformers adapt these mechanisms to effectively handle the interconnected nature of graph data, using both nodes and edges features.

The input graph is fed into a Graph Transformer to generate new contextualized embeddings of its nodes. It is processed through multiple layers. The layer update equations for a layer l are as follows:

$$\hat{h}_i^{l+1} = P_h^l \parallel \left(\sum_{j \in N_i} z_{ij}^{k,l} V^{k,l} h_j^l \right), \quad (15)$$

$$\hat{e}_{ij}^{l+1} = P_e^l \parallel \left(\hat{z}_{ij}^{k,l} \right), \text{ where,} \quad (16)$$

$$z_{ij}^{k,l} = \text{softmax}_j(\hat{z}_{ij}^{k,l}), \quad (17)$$

$$\hat{z}_{ij}^{k,l} = \left(\frac{Q^{k,l} h_i^l \cdot K^{k,l} h_j^l}{\sqrt{d_k}} \right) \cdot E^{k,l} e_{ij}^l, \quad (18)$$

with $Q^{k,l}, K^{k,l}, V^{k,l}, E^{k,l} \in \mathbb{R}^{d_k \times d_{\text{model}}}$, $P_h^l, P_e^l \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}}$, $\parallel$ is the concatenation operation and H is the number of attention heads. As in the original paper, the outputs obtained by exponentiating the terms within the softmax function are constrained within the range of -5 to $+5$ to ensure numerical stability. Subsequently, the resulting values $\hat{h}_i^{l+1}$ and $\hat{e}_{ij}^{l+1}$ are fed into separate Feed Forward Networks (FFN). Residual connections and normalization layers are positioned both before and after the FFN, as:

$$\hat{h}_i^{l+1} = \text{BatchNorm} \left(h_i^l + \hat{h}_i^{l+1} \right), \quad (19)$$

$$h_i^{l+1} = \text{BatchNorm} \left(\hat{h}_i^{l+1} + W_{h,2}^l \text{ReLU}(W_{h,1}^l \hat{h}_i^{l+1}) \right), \quad (20)$$

where $W_{h,1}^l \in \mathbb{R}^{2d_{\text{model}} \times d_{\text{model}}}$, $W_{h,2}^l \in \mathbb{R}^{d_{\text{model}} \times 2d_{\text{model}}}$,

$$\hat{e}_{ij}^{l+1} = \text{BatchNorm} \left(e_{ij}^l + \hat{e}_{ij}^{l+1} \right), \quad (21)$$

$$e_{ij}^{l+1} = \text{BatchNorm} \left(\hat{e}_{ij}^{l+1} + W_{e,2}^l \text{ReLU}(W_{e,1}^l \hat{e}_{ij}^{l+1}) \right), \quad (22)$$

where $W_{e,1}^l \in \mathbb{R}^{2d_{\text{model}} \times d_{\text{model}}}$, $W_{e,2}^l \in \mathbb{R}^{d_{\text{model}} \times 2d_{\text{model}}}$. BatchNorm denotes Batch Normalization [10].

4.3 Scoring each action

Each node has been fed to the graph transformer and has a new contextualized embedding. Let i_κ denote the node containing the vehicle that is about to perform an action. The challenge is to find a way to encode the fact that the vehicle present on node i_κ is the one that the model should be focusing on to get a prediction of the best action to perform. To this end, we concatenate the embedding of i_κ with each of the other nodes, separately (see Figure 2). We get $n - 1$ concatenated pairs of nodes where n is the number of nodes in the graph.

Each of them is fed in parallel to an identical Feed Forward Network with two hidden layers to get a single output number for each pair (i_κ, j) . This number can be interpreted as a measure of the suitability of the concatenated node j as a potential destination for the next action. This approach is similar the work of [11], where the goal is to predict a user-item interaction. In one of the presented settings, the embedding of the user is concatenated with the embedding of the item before being fed to a Feed Forward Network in order to output an interaction score.

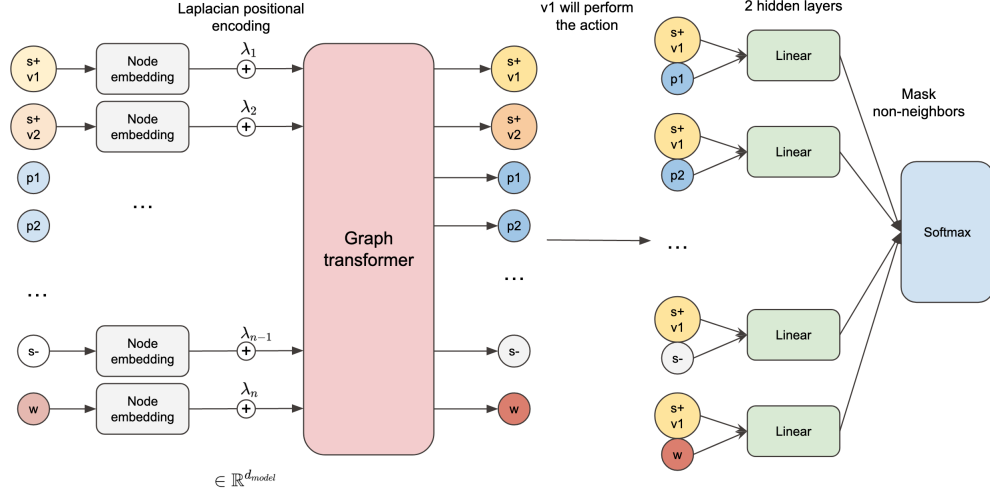


Figure 2: Architecture of the model. We initially pass each node through a linear layer before adding the Laplacian positional encoding to obtain a basic embedding representation. These embeddings are then inputted into a graph transformer, which generates a contextualized version of the node embeddings by considering their relationships within the graph. Next, the node corresponding to the vehicle that is about to perform an action is concatenated with each of the other nodes individually. Each such created pair of nodes is fed into an identical FFN with two hidden layers. The FFN processes the pair of nodes and produces a single output for each pair. The resulting outputs are compared using the softmax function, resulting in a probability distribution over the possible actions.

4.4 Softmax and masks

Every such number is then compared with a softmax layer. Prior to applying softmax, we mask each pair (i_κ, j) that are such that j is not a neighbor of i_κ , so that the probability of choosing this action is 0. The output represents a probability distribution on the action to be performed by the vehicle on i_κ .

4.5 Decoding technique

In a given state, the model outputs a probability distribution on the next possible actions. We can derive a routing schedule by repeatedly sampling the most promising action:

$$a_t = \underset{a}{\operatorname{argmax}} \mathbb{P}(a|s_t). \quad (23)$$

This greedy sampling strategy has a complexity of $\mathcal{O}(n)$ where n is the number of nodes in the graph.

5 Experiments

5.1 Instances

Instances of the DARP are generated following Cordeau’s [1] method, where instances are split into type A and type B. In this work, we focus on type A instances only. The pickup and delivery nodes are generated randomly on a $[-10, 10]^2$ grid. The load of each user’s is set to one, the service duration equals 3, and vehicles have a maximum capacity of 3. The maximum ride time L is set to 30 for all users. Half of the users are considered inbound, meaning their dropoff time window does not have restrictions, and the other half are considered outbound, and their pickup time window is $[0, T]$. The specified time windows have a length of 15 and are generated independently. The routing cost is equal to the Euclidean distance between the nodes. For training, we focus on one of the simplest settings: type A instances with 2 vehicles and 16 users.

5.2 Performance assessment

To assess the performance of the model, we apply it to randomly generated test instances that were not seen during training. Results are compared to the Restricted Fragment (RF) algorithm [3], which solves the mixed integer programming problem with Gurobi [12]. For each test instance, the average routing costs is reported, along with the average gap between the RF solution and our solution. Finally, since our model considers soft constraints, we calculate the number of broken time window and ride time constraints, along with the sum of the margins by which they are violated. In each case, the average is reported. Also, it is interesting to note that the model’s architecture does not depend on the size of the input graph. So although the model is only trained on instances with 2 vehicles and 16 users, we can test it on other instance types to assess the generalization of the model to larger instances. All results are shown in Table 1

5.3 Supervised learning

A training set of 50000 instances was generated. Each instance is solved with the RF algorithm. This generates a dataset of state-action pairs on which the model will be trained. The network embedding dimension for each token was set to $d_{\text{model}} = 512$, each hidden linear layer has a size of $2 \cdot d_{\text{model}}$. The number of graph transformer layers was set to 8, with 16 attention heads each. The key dimension is set to $d_k = 64$. Dropout with $p = 0.1$ is used on each transformer layer. The training set is fed into the model through 28 epochs, with a batch size of 64. The Adam optimizer, with a learning rate of 10^{-4} was used. The loss function is the cross-entropy loss. The training is performed on a NVIDIA V100 PCIe 32GB GPU. The results on 100 test instances are summarized in Table 1.

Instance	Obj	Method	Cost	$\Delta\%$	N_{TW}	N_{RT}	Penalty	Time (s)	B&C Tim
a2-16	271.24	Supervised	277.96	2.67	0.55	1.61	9.54	3.13	0.07
		Reinforcement	292.13	7.84	0.47	0.33	2.98	3.17	
a2-20	337.04	Supervised	346.98	3.07	0.74	1.65	10.8	3.78	0.08
		Reinforcement	358.39	6.48	0.67	0.31	6.34	3.89	
a2-24	402.54	Supervised	429.02	6.7	1.24	2.33	64.5	4.34	0.11
		Reinforcement	435.76	8.37	1.71	0.49	23.94	4.47	
a3-24	371.17	Supervised	412.29	11.21	0.26	1.9	6.07	5.0	0.14
		Reinforcement	438.21	18.23	0.26	0.25	1.54	5.17	
a3-36	547.44	Supervised	635.59	16.2	0.96	2.34	60.74	-	0.26
		Reinforcement	649.51	18.77	1.35	0.42	12.72	-	
a4-32	470.79	Supervised	559.62	18.97	0.22	2.34	8.31	-	0.23
		Reinforcement	599.63	27.52	0.17	0.23	0.59	-	
a4-40	567.43	Supervised	684.03	20.61	0.2	2.7	10.69	-	0.34
		Reinforcement	713.83	25.89	0.31	0.27	2.01	-	
a4-48	676.54	Supervised	840.42	24.29	0.53	2.15	21.13	-	0.45
		Reinforcement	857.97	26.88	0.88	0.57	9.11	-	

Table 1: Supervised and reinforcement results on 100 test instances of the model trained solely on a2-16 instances. Results on other instance types are shown to assess the generalization of the model to larger instances. *Obj* denotes the optimal cost given by the RF algorithm, $\Delta\%$ shows the relative cost difference in percentage, N_{TW} denotes the number of violated time window constraints, N_{RT} denotes the number of violated ride time constraints, while Penalty denotes the sum of the margins by which the constraints were broken. In each case, the average over the 100 instances is reported. In the instance name, a4-32 means type A instance with 4 vehicles and 32 users. For instances larger than a3-24, the input graph wouldn't fit into the GPU, so the evaluation was made on a CPU. Hence, the process time is not reported in those cases since it would not be comparable. The RF algorithm was run on a 8-core AMD Ryzen 7 5800H 3.2GHz CPU with 16GB memory.

Instance	Obj	Cost	$\Delta\%$	N_{TW}	N_{RT}	Penalty	Time (s)	B&C Time (s)
a2-16	271.24	277.96	2.67	0.55	1.61	9.54	3.13	0.07
a2-20	337.04	346.98	3.07	0.74	1.65	10.8	3.78	0.08
a2-24	402.54	429.02	6.7	1.24	2.33	64.5	4.34	0.11
a3-24	371.17	412.29	11.21	0.26	1.9	6.07	5.0	0.14
a3-36	547.44	635.59	16.2	0.96	2.34	60.74	-	0.26
a4-32	470.79	559.62	18.97	0.22	2.34	8.31	-	0.23
a4-40	567.43	684.03	20.61	0.2	2.7	10.69	-	0.34
a4-48	676.54	840.42	24.29	0.53	2.15	21.13	-	0.45

Table 2: Supervised results on 100 test instances of the model trained solely on a2-16 instances. Results on other instance types are shown to assess the generalization of the model to larger instance. $\Delta\%$ shows the relative difference in percentage, N_{TW} denotes the number of broken time window constraints, N_{RT} denotes the number of broken ride time constraints, while Penalty denotes the sum of the margins by which the constraints were broken. In each case, the average over the 100 instances is reported. In the instance name, a3-24 means type A instance with 3 vehicles and 24 users.

5.4 Reinforcement learning

Training the model solely in a supervised learning manner presents certain issues. The model lacks knowledge of any constraints it needs to satisfy; we merely hope that it will learn to replicate the expert policy well enough to avoid violating constraints. However, if the model happens to break a constraint, it will encounter a state that was never seen during training, and optimal performance is not guaranteed. Additionally, supervised learning does not prioritize errors. It is possible that multiple acceptable routing plans exist, and multiple actions could be considered good given a particular state. However, if the model fails to predict the target action, it will be penalized. There is no distinction in the feedback between slightly sub-optimal actions and catastrophic ones.

Both of these issues can potentially be resolved by employing reinforcement learning. The strategy is to allow a partially trained model to generate episodes and provide feedback based on the episode outcomes. Consequently, the model would learn to avoid violating constraints through direct feedback and would not receive excessive penalties for slightly sub-optimal actions. The next sub-sections explain in more details the reinforcement learning implementation. The results on 100 test instances are summarized in Table 1.

5.4.1 Proximal Policy Optimization (PPO)

PPO [13] is a reinforcement learning algorithm that uses an actor and a critic. The actor explores the environment and selects actions. It updates the policy

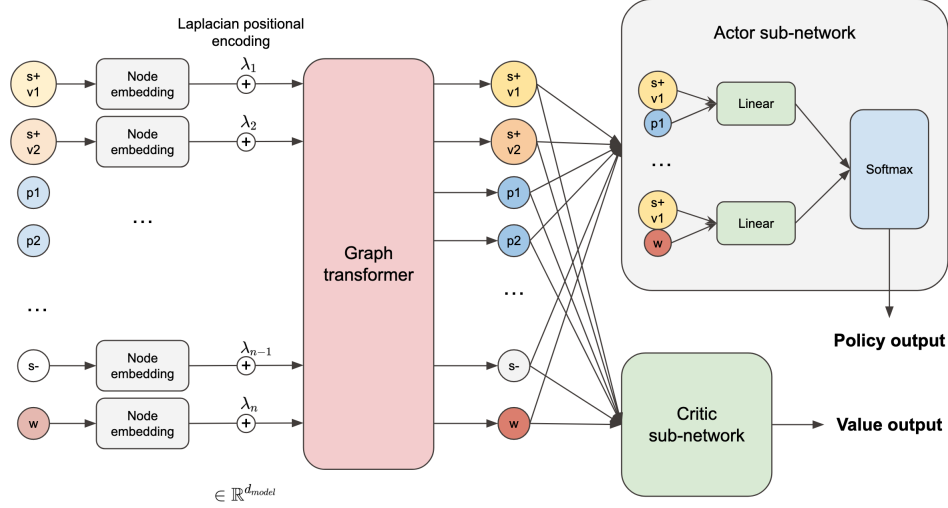


Figure 3: Architecture of the actor-critic model. The actor sub-network is identical to the model of Figure 2. In parallel, a critic sub-network is added. It uses all the nodes embeddings generated by the graph transformer and process them through a FFN with two hidden layers. The initial part up to and including the graph transformer is identical for both the actor and the critic, with shared weights.

based on the critic’s feedback. The critic estimates the value or expected return of being in a particular state, which is used to evaluate the quality of the actor’s actions.

To fit the actor-critic setup, we modify the model shown in Figure 2 to get a new model in Figure 3. The policy network is referred to as the actor and is trained to find an optimal policy. We add a critic sub-network, which is responsible for predicting the cost-to-go from a given state. In other words, it estimates the sum of the total costs until the end of the episode, from a particular state. These two sub-models are trained simultaneously in a multitask learning fashion. Specifically, the initial part of the model up to and including the graph transformer is identical for both the actor and the critic, with shared weights. The outputs of the graph transformer are utilized independently by the actor and the critic.

Both the actor and the critic are first trained in supervised learning. The actor is trained to predict the action given the state as before, while the critic learns to predict the cost-to-go given the state. This supervised training gives a suitable initialization of the weights. The actor and the critic are then further improved using PPO. The reinforcement training consists of multiple steps that are described in Algorithm 1.

5.4.2 Cost computation

After collecting one episode, the cost-to-go from each state s_τ is computed to be used for feedback. It is composed of routing costs and time penalties for violated constraints. The total routing cost $\hat{C}_\tau$ consists of the sum of the immediate routing costs of each action from s_τ until the end of the episode. Let $c(a_i)$ denote the immediate routing cost induced by action a_i at step i , and let $\tilde{\tau}$ denote the last step of the episode, then:

$$\hat{C}_\tau = \sum_{i=\tau}^{\tilde{\tau}} c(a_i) \quad (24)$$

If constraints are broken, we also give a penalty based on the margin by which they are broken. Let $[e_{p_n}, l_{p_n}]$ and $[e_{d_n}, l_{d_n}]$ be the pick-up and the drop-off time windows of user n , t_{p_n} and t_{d_n} be its actual pick up and drop off times, and δ_n be its ride time. For step τ , we penalize for constraint that are broken later in the episode. Let $\mathcal{N}_{P_\tau}$ be the set of users whose pick-up time occurs after step τ , and $\mathcal{N}_{D_\tau}$ be the set of users whose drop-off time occurs after step τ . The time penalty is

$$\begin{aligned} \hat{T}_\tau = & \sum_{n \in \mathcal{N}_{P_\tau}} \left[\max\{e_{p_n} - t_{p_n}, 0\} + \max\{t_{p_n} - l_{p_n}, 0\} \right] \\ & + \sum_{n \in \mathcal{N}_{D_\tau}} \left[\max\{e_{d_n} - t_{d_n}, 0\} + \max\{t_{d_n} - l_{d_n}, 0\} + \max\{\delta_n - \delta_{max}, 0\} \right] \end{aligned} \quad (25)$$

The overall cost-to-go $\hat{R}_\tau$ from each state s_τ is made of the routing cost $\hat{C}_\tau$ and the time penalty $\hat{T}_\tau$ as:

$$\hat{R}_\tau = \hat{C}_\tau + \alpha \hat{T}_\tau \quad (26)$$

where α is a cost ratio hyperparameter.

Algorithm 1: Proximal Policy Optimization

input : Actor π_θ , critic V_θ , learning rate λ , clip rate c , value loss coef α_V , number of epochs N , number of collect episodes n_e , number of update epochs n_u , entropy loss coef α_e , entropy coef decay ϵ .

output : Updated actor π_θ and critic V_θ

- 1 Load the model π_θ, V_θ ;
- 2 **for** $i \leftarrow 1$ **to** N **do**
- 3 $\pi_i, V_i \leftarrow \pi_\theta, V_\theta$;
- 4 Simulate n_e episodes by repeatedly sampling π_i , and save the data;
- 5 Compute actual cost-to-go $\hat{R}_t$ for each (s_t, a_t) ;
- 6 Compute advantage estimates $\hat{A}_t$ for each (s_t, a_t) using Generalized Advantage Estimation (GAE);
- 7 **for** $j \leftarrow 1$ **to** n_u **do**
- 8 **foreach** (s_t, a_t) **do**
- 9 $l_\pi = -\min \left(\frac{\pi_\theta(a_t, s_t)}{\pi_i(a_t, s_t)} A_t, \text{clip} \left(\frac{\pi_\theta(a_t, s_t)}{\pi_i(a_t, s_t)}, 1 - c, 1 + c \right) A_t \right)$;
- 10 $l_V = \text{MSE}(V_\theta(s_t)/\hat{R}_t, 1)$;
- 11 $l_e = -\text{Entropy}(\pi_\theta(s_t, \cdot))$;
- 12 $L = l_\pi + \alpha_V l_V + \alpha_e l_e$;
- 13 Update θ using the overall loss L via gradient descent with Adam;
- 14 $\alpha_e \leftarrow \alpha_e \epsilon$
- 15 **end**
- 16 **end**
- 17 **end**

Instance	Obj	Cost	$\Delta\%$	N_{TW}	N_{RT}	Penalty	Time (s)	B&C Time (s)
a2-16	271.24	292.13	7.84	0.47	0.33	2.98	3.17	0.07
a2-20	337.04	358.39	6.48	0.67	0.31	6.34	3.89	0.08
a2-24	402.54	435.76	8.37	1.71	0.49	23.94	4.47	0.11
a3-24	371.17	438.21	18.23	0.26	0.25	1.54	5.17	0.14
a3-36	547.44	649.51	18.77	1.35	0.42	12.72	-	0.26
a4-32	470.79	599.63	27.52	0.17	0.23	0.59	-	0.23
a4-40	567.43	713.83	25.89	0.31	0.27	2.01	-	0.34
a4-48	676.54	857.97	26.88	0.88	0.57	9.11	-	0.45

Table 3: Reinforcement results on 100 test instances of the model trained solely on a2-16 instances. Results on other instance types are shown to assess the generalization of the model to larger instance. $\Delta\%$ shows the relative difference in percentage, N_{TW} denotes the number of broken time window constraints, N_{RT} denotes the number of broken ride time constraints, while Penalty denotes the sum of the margins by which the constraints were broken. In each case, the average over the 100 instances is reported. In the instance name, a3-24 means type A instance with 3 vehicles and 24 users.

6 Discussion

Given a snapshot of the environment, the presented model trained in supervised learning manages to reproduce single decisions of the Restricted Fragments (RF) algorithm in many cases. However, despite the generated solutions achieving a comparable routing cost as the RF algorithm, they often violate constraints. Another concern arises when the model deviates from a feasible solution, as it makes decisions based on infeasible states that are not included in the training dataset. To address these issues, an attempt was made to train the model using reinforcement learning, which provides direct penalties for violated constraints. This strategy helps reduce the number of violated constraints but slightly increases the overall cost of solutions.

While the model trained on instances with 2 vehicles and 16 users demonstrates reasonable performance in terms of constraint satisfaction when applied to larger instances, there is a significant increase in the gap between the cost of the generated solutions and those provided by the RF algorithm. This disparity is especially important when the number of available vehicles increases. Planning optimal routes with 3 or 4 vehicles appears substantially more complex than planning with 2 vehicles. However, for a fixed number of users, incorporating more vehicles makes it comparatively easier for the model to satisfy the constraints, aligning with intuitive reasoning.

7 Future works

Implementing curriculum learning, where the model is initially trained on small instances and gradually trained on larger instances, could potentially improve the generalization of the model. Also, although the model’s hyperparameters were optimized to some extent through trial and error, further optimization is possible to find a better set of hyperparameters. It is worth noting that the graph input representation used in this study allows for more complex distance computation between pairs of nodes. While the euclidean distance was employed, it is possible to use customized distance computations that consider factors like road networks. Additionally, the architecture of the model is well-suited for addressing the online version of the DARP, which involves a dynamic environment where the number of users can vary over time with new requests coming in. At each time step, the model predicts the next action based solely on the current state. There is no need for consistency between consecutive time steps, and users or vehicles can be dynamically added or removed from the graph. Nevertheless, continued research is required to enable the efficient processing of significantly larger graphs on a GPU, where the potential time-saving advantage of this technique over exact solvers can be realized.

8 Conclusion

This study highlights the potential of deep learning techniques in solving the Dial-a-Ride problem (DARP). The DARP was approached by modeling it as a Markov Decision Process, utilizing a meaningful state representation. The proposed model, based on a variant of the Transformer Encoder architecture, demonstrates effectiveness in handling the complexities and constraints of the DARP. Notably, the model trained on simple instances, such as those with 2 vehicles and 16 users, shows reasonable performance when applied to larger instances. Despite the increased complexity, the model’s ability to generalize indicates its potential to tackle more challenging scenarios.

Comparison of the model trained by supervised learning with a recent Gurobi-based solver shows similar costs, although constraint violations were observed. By incorporating reinforcement learning, the model successfully reduces constraint violations, with a slight increase in solution cost. These findings underscore the significant potential of neural networks in tackling NP-hard problems like the DARP, opening avenues for further advancements in transportation optimization.

References

- [1] Jean-François Cordeau. A branch-and-cut algorithm for the dial-a-ride problem. *Operations Research*, 54(3):573–586, 2006.

- [2] Timo Gschwind and Stefan Irnich. Effective handling of dynamic time windows and its application to solving the dial-a-ride problem. *Transportation Science*, 49(2):335–354, 2015.
- [3] Yannik Rist and Michael A Forbes. A new formulation for the dial-a-ride problem. *Transportation Science*, 55(5):1113–1135, 2021.
- [4] Siddhartha Jain and Pascal Van Hentenryck. Large neighborhood search for dial-a-ride problems. In *International Conference on Principles and Practice of Constraint Programming*, pages 400–413. Springer, 2011.
- [5] Geraldo Regis Mauri, Luiz Antonio, and Nogueira Lorena. Customers’ satisfaction in a dial-a-ride problem. *IEEE Intelligent Transportation Systems Magazine*, 1(3):6–14, 2009.
- [6] Xavier Bresson and Thomas Laurent. The transformer network for the traveling salesman problem. *arXiv preprint arXiv:2103.03012*, 2021.
- [7] Vijay Prakash Dwivedi, Chaitanya K. Joshi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. Benchmarking graph neural networks. 2020.
- [8] Vijay Prakash Dwivedi and Xavier Bresson. A generalization of transformer networks to graphs, 2020.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [10] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift, 2015.
- [11] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering, 2017.
- [12] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2022.
- [13] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017.

Instance	Supervised		Reinforcement		Tabu		SA		RF	
	$\Delta\%$	Time (s)	$\Delta\%$	Time (s)	$\Delta\%$	Time (s)	$\Delta\%$	Time (s)	$\Delta\%$	Time (s)
a3-24	11.21	5.0	18.23	5.17	0.57	5.0	1.19	60	0	0.14
a4-36	-	-	-	-	-	-	-	-	-	-

Table 4:

Algorithm	$\Delta\%$	Time (s)
Supervised	11.21	5.0
Reinforcement	18.23	5.17
Tabu (2018)	0.86	5.0
LNS (2017)	0	79.2
DA (2014)	0	16.6
B&C (2021)	0	0.14

Table 5: Comparison of different algorithms on a3-24 instances. The gap between the produced solutions and the best solutions is reported, along with the processing time.